

FRED TALKS

16th April 2026

CONTENTS

Seeing like an agent: how we design tools in Claude Code

CLAUDE · 1540 WORDS

[Daily article] April 16: Nynetjer

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 505 WORDS

High Amplitude Disagreeableness

STAY SAASY · 931 WORDS

2.1.110

CHANGELOG · 504 WORDS

Make It Myself

XKCD.COM · 32 WORDS

Apr 14, 2026AlignmentAutomated Alignment Researchers: Using
large language models to scale scalable oversight

ANTHROPIC RESEARCH · 1881 WORDS

Seeing like an agent: how we design tools in Claude Code

CLAUDE · 12 APR 2026 · [SOURCE](#)

One of the hardest parts about building an agent harness is constructing its tools.

Claude acts completely through [tool calling](#), but there are a number of ways tools can be constructed in the Claude API with primitives like [bash](#), [skills](#) and [code execution](#). (You can read more about programmatic tool calling on the Claude API in [@RLanceMartin's new article](#)).

So how do you design your agents' tools? Do you give it one general-purpose tool like bash or code execution? Or fifty specialized tools, one for each use case?

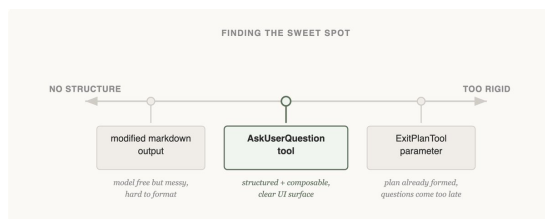
To put yourself in the mind of the model, imagine being given a difficult math problem. What tools would you want in order to solve it? It would depend on your own skill set!

Paper would be the minimum, but you'd be limited by manual calculations. A calculator would be better, but you would need to know how to operate the more advanced options. The fastest and most powerful option would be a computer, but you would have to know how to use it to write and execute code.

This is a useful framework for designing your agent. You want to give it tools that are shaped to its own abilities. But how do you know what those abilities are? You pay attention, read its outputs, experiment. You learn to see like an agent.

If you're building an agent, you'll face the same questions we did: when to add a tool, when to remove one, and how to tell the difference. Here's how we've answered them while building Claude Code, including where we got it wrong first.

Improving elicitation with the AskUserQuestion tool



teacher entirely and instructed the strong model to always choose the most common one. On a coding task, where the model had to predict whether a piece of code was right, the AAR realized it could run the code against some tests and simply read off the right answer. Hacks like these don't invalidate our results (we detected and disqualified these entries), but they clearly do provide a warning. Any deployment of automated researchers will require evaluations that the AARs can't tamper with—and human inspections of both their results and their methods.

To read this research in full, see our [Alignment Science blog](#). The code and datasets for this work are [publicly available, here](#).

But we do think these results have some important implications.

Keeping pace. This study indicates that Claude can meaningfully increase the rate of experimentation and exploration in alignment research. Human researchers can delegate questions to AARs at a very large scale; Claude can take on the task of developing novel hypotheses and iterating on its own results.

Moreover, making progress on weak-to-strong supervision might *itself* help us build more general-purpose Automated Alignment Researchers, which is why we chose this problem for our study. In this study, we frame the weak-to-strong supervision problem as a “crisp” task with a verifiable outcome (increasing the PGR score). We do this because we need a way to automatically and reliably evaluate whether the AAR has made progress. However, if AARs discovered much better weak-to-strong supervision methods that generalized across domains, we could use those same methods to train the AARs to evaluate progress on “fuzzier” tasks that are much harder to verify. (For instance, we could conduct weak-to-strong supervision on Claude’s ability to scope research projects.) This is important, because alignment research—unlike capabilities research—often requires solving much “fuzzier” problems.

Taste and diversity. One possible counter to tools like AARs is that today’s frontier models still lack “research taste” (industry parlance for having an intuitive sense of which ideas might work and which won’t). But the success of AARs in this experiment suggests that the sheer volume of ideas might compensate for a lack of “taste”. If AARs can run many experiments very cheaply, it’s possible they could “brute force” their way into the findings that a very high-taste researcher might’ve come up with, or find success in directions that those researchers might otherwise have given up on.

In turn, this means that the core bottleneck in alignment research could become *evaluation* (making sure that experiments are set up sufficiently well that we’re confident in their results), rather than *generation* (relying on human researchers to propose promising ideas).

Alien science. This work might have some stranger implications, too. AARs, by their nature, are designed to discover ideas that humans might not have considered. But we still need a way to verify whether their ideas and results are sound. For now, we’re still able to interpret what the AARs have done and why. But that might not always be the case: over time, the models’ ideas could become much harder to verify, or corrupted in ways that are tricky for humans to parse or catch. That could mean creating an “alien science”.

Preventing hacks. Even in this highly circumscribed environment, we observed the models “reward hacking”—that is, trying to game our set-up. On math tasks, for instance, one AAR noticed that the most common answer to each problem was *usually* correct, so it skipped the

When building the AskUserQuestion tool, our goal was to improve Claude’s ability to ask questions (often called elicitation).

While Claude could just ask questions in plain text, we found answering those questions felt like they took an unnecessary amount of time. How could we lower this friction and increase the bandwidth of communication between the user and Claude?

Attempt 1: Editing the ExitPlanTool

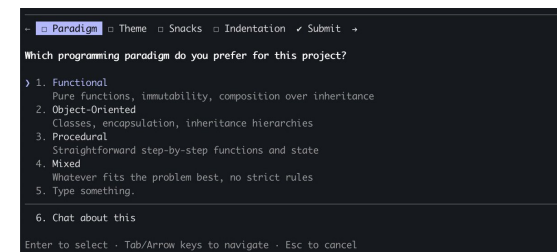
The first approach we tried was adding a parameter to the ExitPlanTool to have an array of questions alongside the plan. This was the easiest fix to implement, but it confused Claude because we were simultaneously asking for a plan and a set of questions about the plan. What if the user’s answers conflicted with what the plan said? Would Claude need to call the ExitPlanTool twice? We knew this tactic wouldn’t work, so we went back to the drawing board. (You can read more about why we made an ExitPlanTool in [our post on prompt caching](#))

Attempt 2: Changing output format

Next, we tried updating Claude’s output instructions to serve a slightly modified markdown format that it could use to ask questions. For example, we could ask it to output a list of bullet point questions with alternatives in brackets. We could then parse and format that question as UI for the user.

Claude could usually produce this format, but not reliably. It would append extra sentences, drop options, or abandon the structure altogether. Onto the next approach.

Attempt 3: The AskUserQuestion Tool



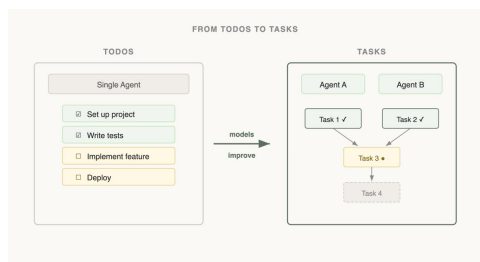
Finally, we landed on creating a tool that Claude could call at any point, but it was particularly prompted to do so during plan mode. When the tool triggered we would show a modal to display the questions and block the agent's loop until the user answered.

This tool allowed us to prompt Claude for a structured output and it helped us ensure that Claude gave the user multiple options. It also gave users ways to compose this functionality, for example calling it in the [Agent SDK](#) or using referring to it in skills.

Most importantly, Claude seemed to like calling this tool and we found its outputs worked well. After all, even the best designed tool doesn't work if Claude doesn't understand how to call it.

Is this the final form of elicitation in Claude Code? We doubt it. As Claude gets more capable, the tools that serve it have to evolve too. The next section shows a case where a tool that once helped started getting in the way.

Updating with capabilities: tasks & todos

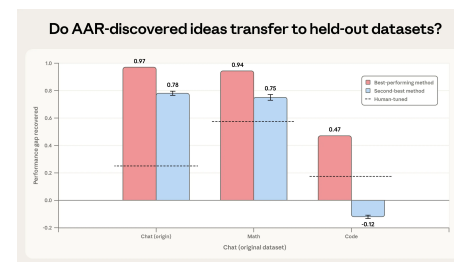


When we first launched Claude Code, we realized that the model needed a [todo list](#) to keep it on track. Todos could be written at the start and checked off as the model did work. To do this we gave Claude the `TodoWrite` tool, which would write or update Todos and display them to the user.

But even then, we often saw Claude forgetting what it had to do. To adapt, we inserted system reminders every 5 turns that reminded Claude of its goal.

As models improved, they found To-do lists limiting. Being sent reminders of the todo list made Claude think that it had to stick to the list instead of modifying it when it realized it needed to change course. We also saw Opus 4.5 also get much better at using subagents, but how could subagents coordinate on a shared todo list?

Seeing this, we replaced the `TodoWrite` feature with [the Task tool](#). Whereas todos are focused on keeping the model on track, tasks help agents communicate with each other. Tasks could include dependencies, share updates across subagents and the model could alter and delete them.



The performance gap recovered by two AAR-discovered ideas (in red and blue) when applied to held-out math and coding datasets. The dashed line indicates the best human-tuned method that we used as a baseline.

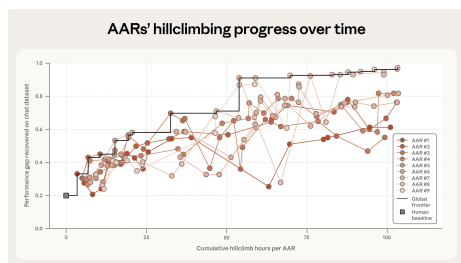
Next, we tested whether the AARs' ideas would work at production scale. We tried out the AARs' most effective method on Claude Sonnet 4 with our production training infrastructure. Here, though, we had less success. The AARs' method didn't lead to a statistically significant improvement. We think this might reflect limitations of this early trial, rather than something more fundamental: our scoring method was quite simple, and we only evaluated a single idea. Nevertheless, this does illustrate a limitation of AARs (at least at their current capabilities): AARs tend to capitalize on opportunities unique to the models and datasets they're given, which means their methods might not work elsewhere. To mitigate this, we suggest allowing AARs to test against multiple domains and datasets during their research. This is one area that future experimentation with AARs could explore.

A few iterations of our experiment taught us more about how to make AARs most effective. For instance, we found that giving each AAR a different starting point helped a lot, even if that starting point was vague. When we tried our experiment *without* setting the AARs off in different directions, they all quickly settled on similar ideas, making much less progress overall (though they still achieved a PGR of almost triple the human baseline). On the other hand, we found that giving the AARs too *much* structure hurt their progress badly. When we prescribed a specific workflow ("propose ideas, then generate a plan, then write the code..."), we found we'd ultimately constrained Claude's work. Left to its own devices, Claude was much more adaptable, designing cheap experiments to test out its ideas before subsequently committing to much more intensive testing.

The success of our AARs in recovering the performance gap between two open-weights models is certainly *not* a sign that frontier AI models are now general-purpose alignment scientists. We deliberately chose a problem that is unusually well-suited to automation, since it has a single, objective measure of success that the models can optimize against. Most alignment problems aren't nearly as neat as this one. And, as we mention below, even in this setting our AARs did their best to game the problem: human oversight remains essential.

To provide a benchmark for the AARs' results, we compared their work to a human baseline. Two of our researchers spent seven days iterating on four of the most promising generalization methods from prior research. On the open-weights models we tested (Qwen 3-4B-Base as the strong model, Qwen 1.5-0.5B-Chat as the weak teacher), the humans recovered 23% of the total performance gap (i.e., achieved a PGR of 0.23).²

Claude improved on this result dramatically. After five further days (and 800 cumulative hours of research), the AARs closed almost the entire remaining performance gap, achieving a final PGR of 0.97. This cost about \$18,000 in tokens and model training expenses, or \$22 per AAR-hour. You can see how each individual AAR progressed from the human baseline (at 0 hours) in the graph below.



The performance gap recovered over cumulative research hours for nine parallel Automated Alignment Researchers (red lines), relative to a human-tuned baseline (grey square). A score of 1.0 means the method fully matches a model trained on ground-truth labels.

Claude, then, did exceptionally well. But how inventive were its methods, and could they be useful in real-world applications? To find out, we ran two further tests.

First, we tested whether the AARs' ideas could recover the performance gap on *held-out* datasets—that is, on tasks that the AARs hadn't already seen. We took the AARs' two highest-performing methods (on a dataset of chat tasks) and applied them to math and coding tasks. Here, our results were relatively promising: the AARs' most effective method successfully generalized to both new datasets, with PGRs of 0.94 on math and 0.47 on coding (which was still double the human baseline). The AARs' second-best method saw mixed results: it worked on math (0.75), but not on code, where it made matters worse. These results suggest that *some* generalizability of the AARs' research is possible, but it isn't a given. We encourage others who try experiments in automated research to stress-test AARs' ideas against held-out datasets, too.

As model capabilities increase, the tools that your models once needed might now be constraining them. It's important to constantly revisit previous assumptions on what tools are needed. This is also why it's useful to stick to a small set of models to support that have a fairly similar capabilities profile.

Designing a search interface

The most consequential tools we've built are the ones that let Claude find its own context.

When Claude Code was first released internally, we used RAG: a vector database would pre-index the codebase, and the harness would retrieve relevant snippets and hand them to Claude before each response.. While RAG was powerful and fast, it required indexing and setup and could be fragile across a host of different environments. Most importantly, Claude was *given* this context instead of finding the context itself.

But if Claude could search on the web, why couldn't it also search your codebase? By giving Claude a Grep tool, we could let it search for files and build context itself.

As Claude gets smarter, it becomes increasingly good at building its context when given the right tools.

When we introduced [Agent Skills](#), we formalized the idea of progressive disclosure, which allows agents to incrementally discover relevant context through exploration.

Claude could now read skill files and those files could then reference other files that the model could read recursively. In fact, a common use of skills is to add more search capabilities to Claude like giving it instructions on how to use an API or query a database.

Over the course of a year, Claude went from not really being able to build its own context to being able to do nested search across several layers of files to find the exact context it needed.

Progressive disclosure is now a common technique we use to add new functionality without adding a tool. In the next section, we explain why.

Progressive disclosure: the Claude Code Guide agent

Claude Code currently has ~20 tools, and our team frequently revisits if we need all of them for Claude to be most effective. The bar to add a new tool is high, because this gives the model one more option to think about.

For example, we noticed that Claude did not know enough about how to use Claude Code. If you asked it how to add a MCP or what a slash command did, it would not be able to reply.

We could have put all of this information in the system prompt, but given that users rarely asked about this, it would have added context rot and interfered with Claude Code’s main job: writing code.

Instead, we tried progressive disclosure: we gave Claude a link to its docs that it could load and search when needed. This worked, but Claude would pull large chunks of documentation into context to find an answer the user could have gotten in one sentence.

So we built the Claude Code Guide — a subagent Claude calls whenever a user asks about Claude Code itself. The subagent does the doc-searching in its own context, follows detailed instructions on how to search and what to extract, and hands back only the answer. The main agent’s context stays clean.

While this isn’t a perfect solution (Claude can still get confused when you ask it about how to set itself up), we were able to add things to Claude’s action space without adding a new tool.

Seeing like an agent is an art, not a science

Designing the tools for your models is as much an art as it is a science. It depends heavily on the model you’re using, the goal of the agent and the environment it’s operating in.

Our best advice? Experiment often, read your outputs, try new things. And most importantly, try to see like an agent.

About the author: *Thariq Shihpar is a member of technical staff at Anthropic, working on Claude Code.*

[Daily article] April 16: Nynetjer

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 16 APR 2026 · [SOURCE](#)

Nynetjer was the third pharaoh of the Second Dynasty of Egypt. Egyptologists have proposed that his reign happened between the late 29th and early 27th century BC and probably lasted around 40 years. Archaeologically, Nynetjer is the best-attested king of the early Second

that extra fine-tuning, which it does by demonstrating what *it* considers ideal outputs to the strong base model. Finally, we evaluate how well the strong model performs after that weak fine-tuning.

In the worst case, the strong model will only be as good as its weak teacher. Ideally, however, the strong model will have learned from the weak teacher’s feedback—it will have interpreted those weak signals in a useful way, using that feedback to improve its performance. We can quantify how well it did so: if the strong model shows no improvement at all (it performs only as well as its weak teacher), we score it 0; if it uses the teacher’s feedback to achieve the ideal outcome—the best performance the strong model could possibly deliver—we score it 1. This measure represents the “performance gap recovered” (between the weak model and the upper limit of the strong model), or the PGR.

As a proxy for scalable oversight, the weak model stands in for humans, and the strong model for the much-smarter-than-human models we might one day need to oversee. If we can make progress on weak-to-strong supervision, we might find that our methods help us keep those ultra-smart models aligned to our values.

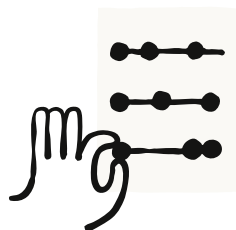
Our new research tests whether Claude can *autonomously* discover ways to improve the PGR. We ask: can Claude develop, test, and analyze alignment ideas of its own? And, if it can, what might that imply about how far today’s AI models can accelerate the pace of alignment research?

To find out, we began with nine copies of Claude Opus 4.6, and gave each one a few extra tools. Each Claude had a place to work and think (that is, a sandbox), a shared forum to circulate its findings with the others, a storage system to upload its code, and a remote server where it could receive a PGR score for each of its ideas. We also provided some background knowledge about model training and inference. We referred to these tooled-up Claude models as Automated Alignment Researchers (or AARs).

To prevent each AAR from pursuing near-identical ideas, we prompted each one with a slightly different (but intentionally ambiguous) starting place: we recommended that one used some interpretability tools, that another thought about reweighting the data in the dataset, and so on.¹ Beyond that, though, we didn’t tell the AARs what to do. It was up to them to propose their own ideas, run their experiments, analyze their results, and share their findings and code with one another in order to work out what to try next.

Apr 14, 2026AlignmentAutomated Alignment Researchers: Using large language models to scale scalable oversight

ANTHROPIC RESEARCH · 14 APR 2026 · [SOURCE](#)



Large language models’ ever-accelerating rate of improvement raises two particularly important questions for alignment research.

One is how alignment can keep up. Frontier AI models are now contributing to the development of their successors. But can they provide the same kind of uplift for *alignment* researchers? Could our language models be used to help align themselves?

A second question is what we’ll do once models become smarter than us. Aligning smarter-than-human AI models is a research area known as “scalable oversight”. Scalable oversight has largely been discussed in theoretical, rather than practical, terms—but at AI’s current pace of improvement, that might not be the case for much longer. For instance, models are already generating vast amounts of code. If their skills progress to the point where they’re generating millions of lines of incredibly complicated code that we can’t parse ourselves, it could become very difficult to tell whether they’re acting in the ways we intend.

In a new Anthropic Fellows study, we pursue both of these questions.

Our new study focuses on a problem known as “weak-to-strong supervision”, a problem that mirrors the one of overseeing smarter-than-human AI models. We start with a relatively strong “base” model—that is, a potentially-capable model that hasn’t yet received fine-tuning to provide its best-possible answers. Then, we use a much *weaker* model as a “teacher” to provide

Dynasty. Most events recorded for his reign on the Palermo Stone are religious festivals and censuses. The probable locations for these events indicate that royal activity was largely confined to the capital Memphis and its vicinity in Lower Egypt. The administrative partition of Egypt into nomes happened during his reign. Nynetjer had a large gallery tomb dug for himself in Saqqara comprising more than 150 rooms, some arranged to model a royal palace. When excavated, the tomb housed funerary equipment and stone tools, some of which were used in a ritual feast for Nynetjer's burial. The subterranean tomb was probably built with associated superstructures, but these were levelled and overbuilt by subsequent pharaohs. (Full article...).

Read more: <https://en.wikipedia.org/wiki/Nynetjer>

Today's selected anniversaries:

556:

Emperor Justinian I approved and consecrated Pope Pelagius I. https://en.wikipedia.org/wiki/Pope_Pelagius_I

1912:

American pilot Harriet Quimby became the first woman to fly across the English Channel. https://en.wikipedia.org/wiki/Harriet_Quimby

1917:

World War I: Several French army corps began a massive assault against the German-occupied Chemin des Dames ridge, south of Laon, France. https://en.wikipedia.org/wiki/Second_Battle_of_the_Aisne

2001:

India and Bangladesh began a six-day conflict over their disputed border, which ended in a stalemate. https://en.wikipedia.org/wiki/2001_Bangladesh%E2%80%93India_border_skirmishes

2013:

Boko Haram militants clashed with government forces, beginning the Baga massacre. https://en.wikipedia.org/wiki/2013_Baga_massacre

Wiktionary's word of the day:

whet: 1. (transitive) 2. To sharpen (something, such as a knife or sword) by rubbing on an object, especially a whetstone; to hone. 3. Of an animal such as a boar: to sharpen (its horns, teeth, tusks, etc.) by rubbing on a stone, etc., in preparation for an attack. 4. (figurative) 5. To make more keen or to stimulate (someone's appetite, interest, etc.); to hone, to sharpen. 6. (obsolete) To encourage (someone) to do something; to incite, to urge. 7. (obsolete) To inculcate or teach (habits, information, etc.). 8. (obsolete, rare) Of a bird: to preen (its feathers). 9. (intransitive, figurative) 10. To make more keen; to stimulate. 11. To prepare for an attack. 12. (obsolete) To consume drink or food as an appetizer. 13. An act of sharpening something by rubbing on an object, especially a whetstone. 14. (by extension, archaic except UK, dialectal) 15. An occasion to do something; a go, a turn. 16. A period of time between two sharpenings of a scythe. 17. (figurative) 18. An item of drink or food consumed as an appetizer or to ward off hunger until a meal; specifically, a small amount of liquor drunk as an appetizer; a dram, a nip. 19. A thing which makes one's desire, interest, etc., more keen; an incitement, an inducement. 20. About Word of the Day 21. Nominate a word 22. Leave feedback <https://en.wiktionary.org/wiki/whet>

Wikiquote quote of the day:

” --Charlie Chaplin https://en.wikiquote.org/wiki/Charlie_Chaplin

High Amplitude Disagreeableness

STAY SAASY · 16 APR 2026 · [SOURCE](#)

True startup people are one of the most important advantages that many tech companies have. Startup people are aggressive, entrepreneurial, and often bring a dynamism that allows them to cut through significant roadblocks. When there's a large platform shift (e.g. the AI wave that is currently occurring), they're often literally the only people at your organization that can help you transition into the new world. There is a reason that many sharp investors strongly prefer to bet on founder-led companies and their startup-oriented teams, particularly during times of extreme change.

But if you're going to make the most of startup people you need to know how to work with them. And if you want to work with startup people, you need to know how to manage their tendencies – including one of their most distinctive traits, which I call high amplitude disagreeableness.

- Fixed headless/SDK session auto-title firing an extra Haiku request when `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` or `CLAUDE_CODE_DISABLE_TERMINAL_TITLE` is set
 - Fixed potential excessive memory allocation when piped (non-TTY) Ink output contains a single very wide line
 - Fixed `/skills` menu not scrolling when the list overflows the modal in fullscreen mode
 - Fixed Remote Control sessions showing a generic error instead of prompting for re-login when the session is too old
 - Fixed Remote Control session renames from `claude.ai` not persisting the title to the local CLI session
-

Make It Myself

XKCD.COM · 15 APR 2026 · [SOURCE](#)



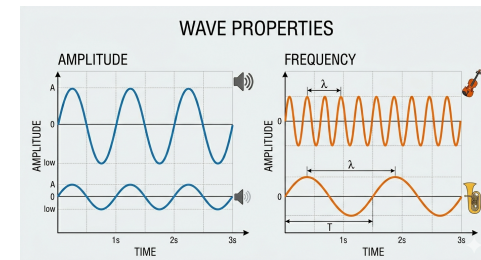
It's not as big a loss as it looks, because now I have have leftover supplies, which will help me talk myself into doing this all over again with a new project!

- Fixed MCP tool calls hanging indefinitely when the server connection drops mid-response on SSE/HTTP transports
- Fixed non-streaming fallback retries causing multi-minute hangs when the API is unreachable
- Fixed session recap, local slash-command output, and other system status lines not appearing in focus mode
- Fixed high CPU usage in fullscreen when text is selected while a tool is running
- Fixed plugin install not honoring dependencies declared in `plugin.json` when the marketplace entry omits them; `/plugin` install now lists auto-installed dependencies
- Fixed skills with `disable-model-invocation: true` failing when invoked via `/` mid-message
- Fixed `--resume` sometimes showing the first prompt instead of the `/rename` name for sessions still running or exited uncleanly
- Fixed queued messages briefly appearing twice during multi-tool-call turns
- Fixed session cleanup not removing the full session directory including subagent transcripts
- Fixed dropped keystrokes after the CLI relaunches (e.g. `/tui`, provider setup wizards)
- Fixed garbled startup rendering in macOS Terminal.app and other terminals that don't support synchronized output
- Hardened "Open in editor" actions against command injection from untrusted filenames
- Fixed `PermissionRequest` hooks returning `updatedInput` not being re-checked against `permissions.deny` rules; `setMode: 'bypassPermissions'` updates now respect `disableBypassPermissionsMode`
- Fixed `PreToolUse` hook `additionalContext` being dropped when the tool call fails
- Fixed stdio MCP servers that print stray non-JSON lines to stdout being disconnected on the first stray line (regression in 2.1.105)

High Amplitude Disagreeableness

Disagreeableness is kind of like a sound wave. Waves have properties of:

- Frequency: In this case, how often someone is disagreeable
- Amplitude: When they actually disagree with something, how intense is the dispute? Do they let things go, or are they righteously upset, insistent, and willing to go all out to make sure that their opinion is followed (or at least fully considered)?



Amplitude vs. Frequency

Some people are basically argumentative and combative all the time: high frequency. Others are actually very genial and (usually) conflict-avoidant: low frequency. That's just human nature.

But *all* startup people share an ability to reach an extremely high amplitude of disagreeableness – if you're really wrong, they will fucking nuke you from orbit, and they are willing to get in front of the entire company and call you an idiot because it doesn't bother them one bit if there's an audience when they push the big red button. One of the defining traits of a startup person is their willingness to disagree publicly, even with senior people, and to remain insistent over a long period of time – for example, fighting to change a bad process or build the right product even if it takes years. When you see a mid-level person politely but firmly disagree with some member of the C-suite in front of a hundred people, you're seeing the startup spirit at work.

What To Do

This pattern of high amplitude disagreeableness among startup people is essentially universal in my experience. As a result, if you want to tap the intensity of an entrepreneurial team, you will need to contend with (or harness) this pattern of behavior. There are a few management implications if you want to attract, retain, or deploy entrepreneurial talent.

First off – what creates this trait? Startup people tend to view their jobs in terms of creation instead of the more typical approach of viewing your job as extraction. If you’re just trying to extract value from a company, it’s really important that people don’t dislike you – and disagreeing vehemently is a pretty quick way to be disliked.

What this means is that if you never seriously disagree with people, even when it’s warranted, startup people won’t respect you. Your inability to reach a high amplitude of disagreeableness is actually an indicator that you aren’t a part of the tribe: You’re an extractor not a creator. *They* are people who are willing to fight for what they believe in with righteous anger; if you aren’t the same, they’ll think that you’re weak at best or lack integrity at worst. Startup people will quit working for these sorts of managers.

Additionally, you can disagree very strongly with startup people, even in public, and they’re surprisingly likely to just brush it off. Big-company corporate types hate getting pushback – it challenges their authority, and the perception of authority is how you climb the corporate ladder. High amplitude disagreement destabilizes existing power structures because it levels the playing field – it lets people step out of their lane to make change as long as they have enough conviction. Startup types don’t exactly *like* being challenged, but they recognize it as necessary at times, and a sign that someone actually cares. You can often recognize a healthy entrepreneurial culture because it accepts people being highly disagreeable (within reason) without permanently branding them as a troublemaker.

You also need to have a culture where strong disagreement doesn’t lead to punishment or ostracism. This can be surprisingly challenging. If you manage entrepreneurial startup types, you will find yourself explaining, defending, or occasionally apologizing for the fact that they very intensely disagreed with someone else surprisingly often. It’s essential that you create a culture that can accommodate the fact that there are going to be some intense arguments, while also making sure that your workplace stays professional and pleasant.

Finally, startup people really tend to hold grudges if you’re confidently wrong. At the root of high amplitude disagreeableness is a belief that the truth is knowable and that it matters, deeply. Startup people are running software where their conviction meter maxes out at a much higher level. This means that if you’re wrong in big ways, and you don’t correct it, startup people are guaranteed to remember and hold it against you since they believe that the difference between right and wrong really matters. Corporate types might hold their noses and put up with outright incorrectness, but startup people won’t. If you want to lose the startup people on your team, the most reliable way is to be confidently wrong.

2.1.110

CHANGELOG · 15 APR 2026 · [SOURCE](#)

- Added `/tui` command and `tui` setting — run `/tui fullscreen` to switch to flicker-free rendering in the same conversation
- Changed `Ctrl+O` to toggle between normal and verbose transcript only; focus view is now toggled separately with the new `/focus` command
- Added `autoScrollEnabled` config to disable conversation auto-scroll in fullscreen mode
- Added option to show Claude's last response as commented context in the `Ctrl+G` external editor (enable via `/config`)
- Improved `/plugin` Installed tab — items needing attention and favorites appear at the top, disabled items are hidden behind a fold, and `f` favorites the selected item
- Improved `/doctor` to warn when an MCP server is defined in multiple config scopes with different endpoints
- `--resume/--continue` now resurrects unexpired scheduled tasks
- `/autocompact`, `/context`, `/exit`, and `/reload-plugins` now work from Remote Control (mobile/web) clients
- Write tool now informs the model when you edit the proposed content in the IDE diff before accepting
- Bash tool now enforces the documented maximum timeout instead of accepting arbitrarily large values
- SDK/headless sessions now read `TRACEPARENT/TRACESTATE` from the environment for distributed trace linking
- Session recap is now enabled for users with telemetry disabled (Bedrock, Vertex, Foundry, `DISABLE_TELEMETRY`). Opt out via `/config` or `CLAUDE_CODE_ENABLE_AWAY_SUMMARY=0`.